PORTFOLIO – PART 1: CONCEPTION PHASE

Task 1: Host a Highly Available, Globally Accessible Webpage on AWS

1. CONCEPT & OBJECTIVE

The aim of this work is to design the cloud architecture for a company website, here represented by a simple “Hello World” HTML page, and to provision it later as a robust, secure and highly available web server environment on Amazon Web Services (AWS). The planned architecture, named **“GlobalEdge-HA Web Stack”**, follows the reliability, performance-efficiency and security pillars of the AWS Well-Architected Framework (AWS, 2024) and is described completely as Infrastructure as Code (IaC) with Terraform. Four goals guide every design decision: (1) **low latency** for visitors from all around the globe by serving content from the edge location nearest to them; (2) **high availability**, so that the failure of a single server or even an entire data centre causes no downtime; (3) **elasticity**, so that the backend scales out automatically during traffic spikes and scales in again to save costs; and (4) **replicability**, so that the whole environment can be created, reviewed, reproduced and destroyed with a few commands. The procedure is to (a) define the components and their interaction in this concept, (b) write modular Terraform code (network, load balancing, compute, CDN/DNS) in phase 2, and (c) test failover and scaling before finalisation.

2. ARCHITECTURAL DECISIONS & SERVICE JUSTIFICATIONS

- **Global latency – Amazon Route 53 & Amazon CloudFront:** Route 53 is the authoritative DNS service and resolves the domain through an alias record to the CloudFront distribution, using anycast and health checks with a 100 % availability SLA. CloudFront caches the static Hello World assets (HTML, CSS, images) at more than 600 global edge locations, so most requests never travel to the origin region. HTTPS is terminated at the edge (ACM certificate, AWS Shield Standard against DDoS); cache misses go to the ALB origin in *eu-central-1*.
- **High availability – Application Load Balancer across Multi-AZ:** A custom VPC (10.0.0.0/16) spans two Availability Zones (AZ A and AZ B), each with one public and one private subnet. An Internet Gateway (IGW) attaches the VPC to the internet; the internet-facing ALB lives in both public subnets, performs health checks and only forwards traffic to healthy targets in the **private subnets**. One NAT Gateway per AZ gives the private servers outbound-only access (patches, packages). Security groups form the security boundaries: the ALB accepts HTTP/HTTPS only from the CloudFront managed prefix list, and the EC2 instances accept port 80 only from the ALB security group.
- **Backend autoscaling – EC2 Auto Scaling Group:** A launch template (Amazon Linux 2023, *t3.micro*, *user_data* installing a lightweight **Nginx** configuration) feeds an ASG distributed across both private subnets (min 2 / desired 2 / max 6). A target-tracking policy driven by Amazon CloudWatch keeps average CPU utilisation at about 50 % and additionally scales on ALB request count per target, so servers are provisioned during traffic spikes and terminated when demand falls. Unhealthy instances are replaced automatically. An Amazon S3 bucket (private VPC gateway endpoint) stores shared media and the Terraform remote state, keeping servers stateless.
- **Infrastructure as Code – Terraform:** Terraform (HashiCorp, 2024) was chosen over AWS CloudFormation because of its readable declarative HCL syntax, its *plan* preview before changes, reusable modules and its provider-agnostic ecosystem. All resources are versioned in Git, the state is stored in S3 with locking, and *terraform apply* / *terraform destroy* make the environment reliably reproducible and easy to dismantle, avoiding idle costs. *Alternatives considered:* an S3 static website behind CloudFront would be cheaper but cannot demonstrate backend autoscaling; ECS Fargate or Elastic Beanstalk would add container or platform overhead that is not necessary for a single web page.

3. AWS CLOUD ARCHITECTURE DIAGRAM

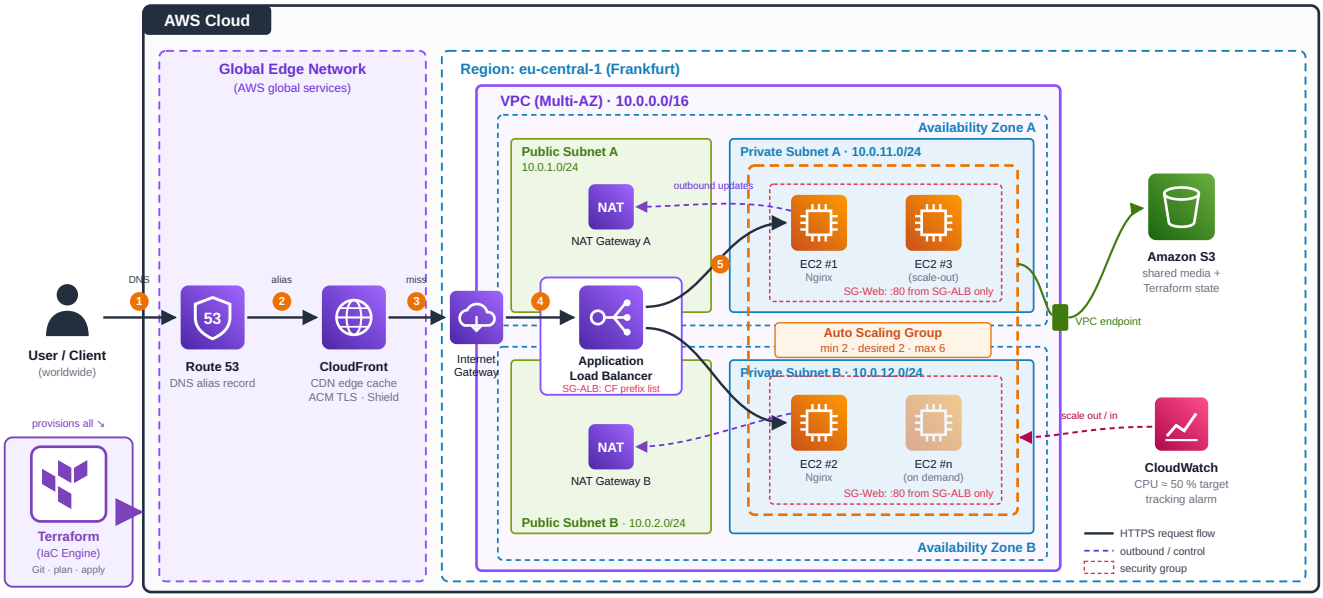


Figure 1: “GlobalEdge-HA Web Stack” – request path from the global user via Route 53 and CloudFront into a Multi-AZ VPC (ALB in public subnets, EC2 Auto Scaling Group in private subnets), provisioned end-to-end by Terraform (own illustration).

References: AWS (2024). *AWS Well-Architected Framework – Reliability Pillar*. https://docs.aws.amazon.com/wellarchitected/ · AWS (2024). *Amazon CloudFront Developer Guide*. https://docs.aws.amazon.com/cloudfront/ · HashiCorp (2024). *Terraform AWS Provider Documentation*. https://registry.terraform.io/providers/hashicorp/aws/latest/docs